



Department of Artificial Intelligence and Machine Learning

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Course Code : AI253IA

Date : 31/1/2024

Semester : V Sem

Time : 9:30am to 11:30am

Max Marks : 10 (Q) + 50(CIE)

Duration : 30 + 90=120 min

QUIZ

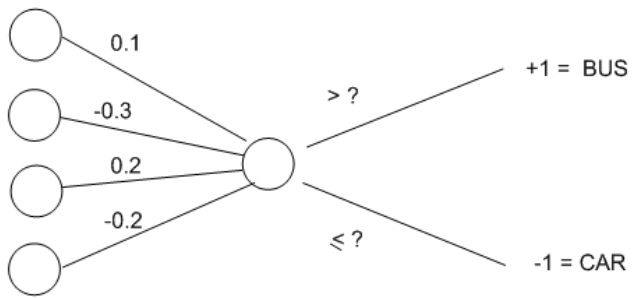
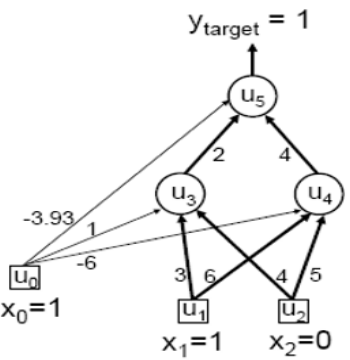
Note: Answer all the Questions

Q. No	Questions	M	BT	CO
1	a) Consider a gradient descent-based learning rate annealing with a learning rate. The initial learning rate is 0.1 and the learning rate decays with a rate parameter of 0.5 per iteration. Calculate the learning rate after the 5 th and 10 th iteration	2	3	3
	b) List two stopping criteria of Back proportion algorithm	2	1	1
	c) In which scenarios might sequential learning be preferred over batch learning , and why?	2	4	2
	d) Define Delta rule for weight updating	2	2	1
	e) In memory-based learning, what is the main technique used to make predictions?	2	1	1

CIE

Note: Answer all the Questions

Q. No	Questions	M	BT	CO
2	a) Compare the following activation functions: threshold, sigmoid, and piecewise linear functions. Include their mathematical expressions and graphs.	5	4	2
	b) With neat diagram briefly explain non linear mathematical model of Neuron and mention the importance of bias.	5	2	1
3	a) Consider an adaptive filter designed to remove noise from a desired signal d(n). The input signal x(n) to the filter is a combination of the signal s(n) and a noise v(n), such that x(n)=s(n)+v(n) a) Define the adaptive filter's objective and the role of the error signal e(n). b) Briefly explain how the Least Mean Squares (LMS) algorithm updates the filter coefficients to minimize the error	5	3	3
	b) Describe Hebb's learning rule and its limitations and explain how the covariance hypothesis addresses the limitations of Hebb's learning	5	2	1
4	a) A competitive learning network has three neurons with initial weights w1=0.3, w2=0.5, and w3=0.2. For an input x1=0.4, x2=0.6, x3=0.8, draw fully connected architecture and identify the winning neuron	5	3	3
	b) Consider the cost function e(w), where d ² is some constant, and input value (x) is 1, find the optimum value of w for which the given cost function reaches its minimum value $e(w) = \frac{1}{2}d^2 - r^T xw + \frac{1}{2}w^T R w$	5	3	3

		$R_x = \begin{bmatrix} 1 & 0.8182 \\ 0.8182 & 1 \end{bmatrix}$ $r = \begin{bmatrix} 0.8182 \\ 0.354 \end{bmatrix}$			
5	a)	Suppose we are training a neural network to learn a function that takes three integers as input, representing weight, fuel capacity, and passenger numbers, and outputs either “car” or “bus”. Suppose the ANN currently looks like shown in Fig. 2b.  Fig: 2b. - Does the network predict a bus or a car for this triple of inputs: (1, 10, 12, 13)? - Suppose that the triple (1, 10, 12, 13) has been miscategorized by the perceptron. Using the perceptron learning rule, calculate the weight change for the weights in the network in light of this training network, if we use a learning rate of 0.1. - What does the re-trained network look like? - Does the re-trained network (feed-forward) correctly categorize the example (10, 12, 13)? Has the network over-corrected? This triple: (5,7,7) used to be (correctly) categorized as a car. Is it still correctly categorized?	7	4	2
	b)	The Correlation Matrix R_x of the input vector $x(n)$ in the LMS algorithm is defined by $R_x = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$ Define the range of the values for the learning rate parameter n of the LMS algorithm for it to be convergent in the mean square	3	3	3
6	a)	The network weights have been initialized as shown in Fig.2a. Analyze the Multilayer perceptron Back Propagation algorithm for the initialized network by doing the following (use the sigmoid function): - Compute the output of hidden layers and output layer. - Calculate the error and delta values. - Find the new weights and show how the new error is reduced (Up to 2 iterations) 	1 0	5	4

M-Marks, BT-Blooms Taxonomy Levels, CO-Course Outcomes

	Particulars	CO1	CO2	CO3	CO4	L1	L2	L3	L4	L5	L6
--	-------------	-----	-----	-----	-----	----	----	----	----	----	----

ANNDL CIE 1 - Quiz Solutions

Date: 31/1/2024

Subject: Artificial Neural Networks and Deep Learning

Assessment: CIE 1 (Quiz)

1 a) Learning Rate Annealing Calculation

Question: Consider a gradient descent-based learning rate annealing with a learning rate. The initial learning rate is 0.1 and the learning rate decays with a rate parameter of 0.5 per iteration. Calculate the learning rate after the 5th and 10th iteration.

Solution:

Learning rate annealing (or decay) is used to gradually reduce the learning rate over time, allowing the algorithm to settle into a minimum. The standard formula for time-based learning rate decay is:

$$\eta_n = \frac{\eta_0}{1 + k \cdot n}$$

Where:

- η_n = Learning rate at iteration n
- η_0 = Initial learning rate (0.1)
- k = Decay rate parameter (0.5)
- n = Iteration number

i. After the 5th iteration ($n = 5$):

$$\eta_5 = \frac{0.1}{1 + (0.5 \times 5)}$$

$$\eta_5 = \frac{0.1}{1 + 2.5}$$

$$\eta_5 = \frac{0.1}{3.5}$$

$$\eta_5 \approx \mathbf{0.0286}$$

ii. After the 10th iteration ($n = 10$):

$$\eta_{10} = \frac{0.1}{1 + (0.5 \times 10)}$$

$$\eta_{10} = \frac{0.1}{1 + 5}$$

$$\eta_{10} = \frac{0.1}{6}$$

$$\eta_{10} \approx \mathbf{0.0167}$$

1 b) Stopping Criteria of Back Propagation

Question: List two stopping criteria of Back propagation algorithm.

Solution:

The Backpropagation algorithm is an iterative process. Common criteria used to decide when to stop training include:

1. **Gradient Threshold:** Stop when the magnitude of the gradient vector becomes sufficiently small (e.g., below a small threshold ϵ). This indicates the algorithm has reached a local minimum or a flat plateau.
 - *Criterion:* $\|\nabla E(w)\| < \epsilon$
2. **Absolute Error Convergence:** Stop when the total mean squared error (MSE) over the training set falls below a pre-defined acceptable level.
 - *Criterion:* $E_{total} < \epsilon$
3. **Generalization / Early Stopping:** Stop when the error on a separate **validation dataset** starts to increase, even if the training error is still decreasing. This prevents overfitting.
4. **Maximum Epochs:** Stop after a fixed number of iterations (epochs) have been completed to prevent infinite loops if convergence is not reached.

1 c) Sequential vs. Batch Learning

Question: In which scenarios might sequential learning be preferred over batch learning, and why?

Solution:

Sequential Learning (also known as Online Learning or Stochastic Gradient Descent) updates weights after seeing a single training example, whereas **Batch Learning** updates weights only after processing the entire dataset.

Preferred Scenarios for Sequential Learning:

1. **Large Datasets / Memory Constraints:**

- *Why:* When the dataset is too large to fit into the computer's RAM, batch learning is computationally infeasible. Sequential learning only requires one sample in memory at a time.

2. Real-time / Dynamic Environments:

- *Why:* If data is arriving in a continuous stream (e.g., sensor data, stock prices), the model needs to adapt on the fly. Sequential learning can update the model instantly without waiting for a full "batch" to accumulate.

3. Redundancy in Data:

- *Why:* If the dataset contains many similar examples, batch learning wastes computation calculating gradients for all of them before one update. Sequential learning can converge faster by taking many steps with similar information.

4. Escaping Local Minima:

- *Why:* The noise introduced by updating weights based on single examples (stochastic behavior) can help the algorithm "jump" out of shallow local minima, whereas the smooth path of batch learning might get stuck.

1 d) Delta Rule

Question: Define Delta rule for weight updating.

Solution:

The **Delta Rule** (also known as the Widrow-Hoff rule or LMS rule) is a gradient descent learning rule for updating the weights of the inputs to artificial neurons. It states that the adjustment to a weight should be proportional to the product of the input signal and the error signal (the difference between the target and the actual output).

The Mathematical Definition: For a weight w_{ji} connecting input unit i to output unit j :

$$\Delta w_{ji} = \eta \cdot (t_j - y_j) \cdot x_i$$

$$w_{ji}^{new} = w_{ji}^{old} + \Delta w_{ji}$$

Where:

- Δw_{ji} is the change in weight.
- η (eta) is the learning rate (a small positive constant).
- t_j is the target (desired) output for neuron j .
- y_j is the actual computed output of neuron j .
- $(t_j - y_j)$ represents the error signal (δ).
- x_i is the input value from presynaptic neuron i .

In summary: **Weight Update = Learning Rate × Error × Input**

1 e) Memory-based Learning Technique

Question: In memory-based learning, what is the main technique used to make predictions?

Solution:

The main technique used in memory-based learning (also called instance-based learning or lazy learning) is the **Nearest Neighbor** approach (specifically **k-Nearest Neighbors** or **k-NN**).

Mechanism: Instead of learning a fixed set of parameters (like weights in a neural network) during a training phase, the algorithm simply **stores (memorizes) the training data**.

To make a prediction for a new query instance:

1. It calculates the distance (e.g., Euclidean distance) between the new instance and the stored instances.
2. It retrieves the "nearest" stored instances.
3. It predicts the output based on the majority class (classification) or average value (regression) of these neighbors.

Test: 2

2 a) Activation Functions Comparison

Question: Compare the following activation functions: threshold, sigmoid, and piecewise linear functions. Include their mathematical expressions and graphs.

Solution:

Activation functions determine the output of a neural network node given an input or set of inputs. They introduce non-linear properties to the network.

1. Threshold Function (Heaviside Step Function)

This is a binary classifier. It activates the neuron only if the input sums to a value greater than or equal to 0 (or a specific threshold).

- **Mathematical Expression:**

$$\varphi(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{if } v < 0 \end{cases}$$

- **Characteristics:**

- Output is binary (0 or 1).
- Not differentiable at $v = 0$ (cannot be used with standard Backpropagation).
- **Graph:** It looks like a step. It is flat at 0 for negative inputs and instantly jumps to 1 for positive inputs.

2. Sigmoid Function (Logistic Function)

This is an S-shaped curve that maps the input to a value between 0 and 1. It is strictly increasing and continuous.

- **Mathematical Expression:**

$$\varphi(v) = \frac{1}{1 + e^{-av}}$$

(Where a is the slope parameter. As $a \rightarrow \infty$, it approaches the threshold function).

- **Characteristics:**
 - Output range is $(0, 1)$.
 - It is differentiable everywhere, making it suitable for gradient-based learning (Backpropagation).
 - **Graph:** A smooth 'S' curve. It starts near 0, rises steeply near the origin, and saturates near 1.

3. Piecewise Linear Function

This function approximates a non-linear function with linear segments. A common form saturates at 0 and 1 but is linear in between.

- **Mathematical Expression:**

$$\varphi(v) = \begin{cases} 1 & \text{if } v \geq +1/2 \\ v + 1/2 & \text{if } -1/2 < v < +1/2 \\ 0 & \text{if } v \leq -1/2 \end{cases}$$

- **Characteristics:**
 - Linearly increases within a specific amplification range.
 - Saturates outside this range.
 - **Graph:** Flat at 0, then a straight diagonal line going up, then flat at 1.

Feature	Threshold	Sigmoid	Piecewise Linear
Output Range	{0, 1}	(0, 1)	[0, 1]
Differentiable	No	Yes	Almost everywhere
Usage	Perceptrons (Binary classification)	Hidden layers, Probability estimation	Approximations

2 b) Non-Linear Mathematical Model of a Neuron

Question: With neat diagram briefly explain non linear mathematical model of Neuron and mention the importance of bias.

Solution:

An artificial neuron (McCulloch-Pitts model) is the fundamental building block of a neural network. It mimics the function of a biological neuron by processing input signals to produce an output.

Components of the Model:

1. **Synapses (Input Links):** Each input signal x_j is connected to the neuron k via a synaptic weight w_{kj} .
 - Specifically, a signal x_j at input j is multiplied by weight w_{kj} .
2. **Summing Junction (Linear Combiner):** Sums up the weighted inputs.
 - Induced local field $v_k = \sum_{j=1}^m w_{kj}x_j$.
3. **Bias (b_k):** An external parameter that acts effectively as an input fixed at $+1$ with its own weight. It is added to the weighted sum.
4. **Activation Function ($\varphi(\cdot)$):** A non-linear function that defines the output of the neuron in terms of the induced local field.

Mathematical Representation:

$$u_k = \sum_{j=1}^m w_{kj}x_j$$

(Linear weighted sum)

$$v_k = u_k + b_k$$

(Adding bias)

$$y_k = \varphi(v_k)$$

(Final Output)

Diagram Description:

Imagine a structure where multiple inputs $(x_1, x_2, \dots, x_m)$ flow into a central node.

1. Each line carrying an input x passes through a "multiplier" block labeled w .
2. All these weighted lines point into a circle labeled Σ (Summing Junction).
3. Another line labeled "Bias (b_k)" also points into Σ .
4. The output of Σ (which is v_k) goes into a block labeled $\varphi(\cdot)$ (Activation Function).
5. The arrow coming out of $\varphi(\cdot)$ is the final output y_k .

The Importance of Bias:

The bias b_k is critical because it allows the activation function to be shifted to the left or right along the horizontal axis.

- **Without bias:** The hyperplane (decision boundary) separating classes must pass through the origin $(0, 0)$. This severely limits the model's flexibility.
- **With bias:** The model can fit data patterns that are not centered at zero. It acts like the

ANNDL CIE 1 - Quiz Solutions (Part 2)

Subject: Artificial Neural Networks and Deep Learning **Questions:** 3 and 4

3 a) Adaptive Filter and LMS Algorithm

Question: Consider an adaptive filter designed to remove noise from a desired signal $d(n)$. The input signal $x(n)$ to the filter is a combination of the signal $s(n)$ and a noise $v(n)$, such that $x(n) = s(n) + v(n)$. a) Define the adaptive filter's objective and the role of the error signal $e(n)$. b) Briefly explain how the Least Mean Squares (LMS) algorithm updates the filter coefficients to minimize the error.

Solution:

a) Objective and Role of Error Signal

- **Objective:** The primary objective of the adaptive filter is to adjust its transfer function (defined by its weights/coefficients) automatically to minimize the difference between its output $y(n)$ and a desired response $d(n)$. In the context of noise removal (filtering), the goal is to produce an output $y(n)$ that closely approximates the clean signal component $s(n)$ by filtering the noisy input $x(n)$. Ideally, the filter learns to pass the signal frequencies and attenuate the noise frequencies.
- **Role of Error Signal $e(n)$:** The error signal is defined as the difference between the desired signal and the filter output:

$$e(n) = d(n) - y(n)$$

The error signal serves as the **feedback mechanism** or "teacher" for the adaptive process. It quantifies performance; a large error indicates the weights are far from optimal, while a small error indicates convergence. The adaptive algorithm uses $e(n)$ to calculate the direction and magnitude of the weight adjustments needed for the next iteration.

b) LMS Algorithm Update Mechanism

The Least Mean Squares (LMS) algorithm is a stochastic gradient descent method. It updates the filter coefficients (weights) iteratively to minimize the instantaneous squared error $e^2(n)$, which acts as a proxy for the Mean Squared Error (MSE), $J(w) = E[e^2(n)]$.

The Update Rule (Delta Rule):

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n)$$

Explanation:

1. **Calculate Output:** The filter computes the output $y(n) = w^T(n)x(n)$.
2. **Calculate Error:** The error $e(n) = d(n) - y(n)$ is computed.

3. **Update Weights:** The weights are adjusted by adding a term proportional to the product of the error and the input vector.
 - μ (learning rate) controls the step size.
 - The term $-\nabla(e^2(n)) \approx 2e(n)x(n)$ represents the estimated negative gradient of the error surface.
 - By moving in the direction of the negative gradient, the filter descends the error surface towards the minimum MSE (the Wiener solution).

3 b) Hebb's Learning Rule and Covariance Hypothesis

Question: Describe Hebb's learning rule and its limitations and explain how the covariance hypothesis addresses the limitations of Hebb's learning.

Solution:

Hebb's Learning Rule: Proposed by Donald Hebb (1949), this is the oldest learning rule, summarized by the phrase: "*Cells that fire together, wire together.*" It states that if a neuron i receives an input from neuron j and both are highly active simultaneously, the synaptic weight w_{ij} connecting them should increase.

- **Mathematical Formula:**

$$\Delta w_{ij} = \eta \cdot y_i \cdot x_j$$

Where η is the learning rate, y_i is the output of the post-synaptic neuron, and x_j is the input from the pre-synaptic neuron.

Limitations:

1. **Instability (Unbounded Growth):** Since firing rates (y_i, x_j) are typically positive, the weights only increase ($\Delta w_{ij} > 0$). Without a mechanism to decrease weights, they grow indefinitely, leading to saturation and instability.
2. **No Depression:** It cannot account for Long-Term Depression (LTD), where synaptic strength decreases when activity is uncorrelated.

Covariance Hypothesis: The covariance rule (Sejnowski, 1977) addresses these limitations by using the **deviation from the mean** activity levels rather than raw activity levels.

- **Mathematical Formula:**

$$\Delta w_{ij} = \eta \cdot (y_i - \bar{y}) \cdot (x_j - \bar{x})$$

Where $\bar{y}$ and $\bar{x}$ are the average (expected) values of the output and input, respectively.

- **How it addresses limitations:**
 1. **Allows Weight Decrease:** If the pre-synaptic and post-synaptic activities are anti-correlated (e.g., input is high while output is low), the product becomes negative, causing the weight to decrease (LTD).

2. **Stability:** Over time, if activities fluctuate around their means, the net weight change tends to balance out, preventing unbounded growth. It learns the covariance between input and output.

4 a) Competitive Learning Network

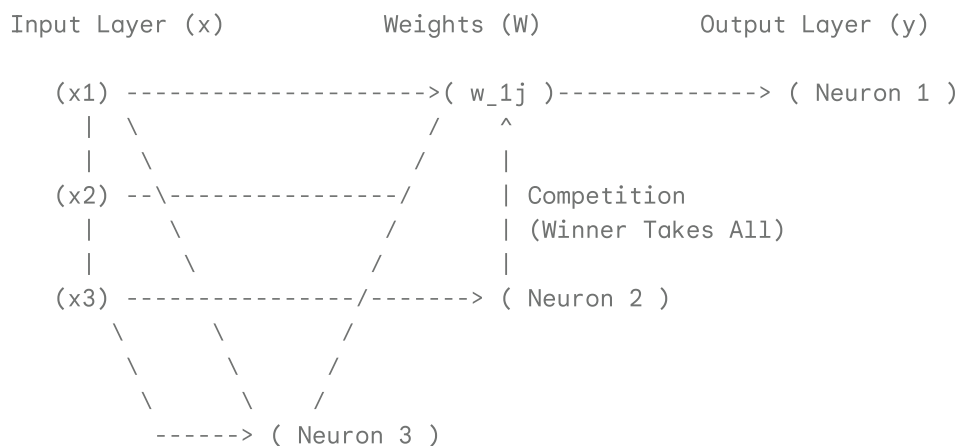
Question: A competitive learning network has three neurons with initial weights $w_1 = 0.3$, $w_2 = 0.5$, and $w_3 = 0.2$. For an input $x_1 = 0.4$, $x_2 = 0.6$, $x_3 = 0.8$, draw fully connected architecture and identify the winning neuron.

Solution:

Architecture Assumption: The problem specifies "Three neurons" (Output Layer) and "For an input x_1, x_2, x_3 " (Input Layer). A fully connected architecture implies every input node connects to every output neuron.

- **Input Vector:** $\mathbf{x} = [0.4, 0.6, 0.8]^T$.
- **Weights:** The problem provides one scalar weight per neuron (w_1, w_2, w_3). In a fully connected network, each neuron requires a weight vector. We assume the provided scalar represents the initial value for *all* elements of that neuron's weight vector (i.e., $\mathbf{w}_1 = [0.3, 0.3, 0.3]^T$).

1. Fully Connected Architecture Diagram:



(Note: In a true drawing, lines would connect every x to every Neuron. Neuron 1 has weight vector $\mathbf{w}_1$, Neuron 2 has $\mathbf{w}_2$, etc.)

2. Identifying the Winning Neuron: In competitive learning, the "winning" neuron is the one whose weight vector is most similar (closest) to the input vector. We commonly use **Euclidean Distance** ($D = ||\mathbf{x} - \mathbf{w}||$). The winner has the *minimum* distance.

Input Vector $\mathbf{x} = [0.4, 0.6, 0.8]$

- **Neuron 1:** $\mathbf{w}_1 = [0.3, 0.3, 0.3]$ (Assuming uniform initialization from scalar)

$$D_1^2 = (0.4 - 0.3)^2 + (0.6 - 0.3)^2 + (0.8 - 0.3)^2$$

$$D_1^2 = (0.1)^2 + (0.3)^2 + (0.5)^2 = 0.01 + 0.09 + 0.25 = \mathbf{0.35}$$

- **Neuron 2:** $\mathbf{w}_2 = [0.5, 0.5, 0.5]$

$$D_2^2 = (0.4 - 0.5)^2 + (0.6 - 0.5)^2 + (0.8 - 0.5)^2$$

$$D_2^2 = (-0.1)^2 + (0.1)^2 + (0.3)^2 = 0.01 + 0.01 + 0.09 = \mathbf{0.11}$$

- **Neuron 3:** $\mathbf{w}_3 = [0.2, 0.2, 0.2]$

$$D_3^2 = (0.4 - 0.2)^2 + (0.6 - 0.2)^2 + (0.8 - 0.2)^2$$

$$D_3^2 = (0.2)^2 + (0.4)^2 + (0.6)^2 = 0.04 + 0.16 + 0.36 = \mathbf{0.56}$$

Conclusion: Comparing the squared distances: $0.11 < 0.35 < 0.56$. **Neuron 2 is the winning neuron.**

4 b) Optimum Weight Value Calculation

Question: Consider the cost function $e(w) = \frac{1}{2}d^2 - r^T w + \frac{1}{2}w^T R w$ (assuming $x = 1$ in the middle term as per standard quadratic form). Find the optimum value of w for which the cost function reaches its minimum. Given:

$$R = \begin{bmatrix} 1 & 0.8182 \\ 0.8182 & 1 \end{bmatrix}, \quad r = \begin{bmatrix} 0.8182 \\ 0.354 \end{bmatrix}$$

Solution:

1. Condition for Minimum: To find the optimum weight vector w_{opt} that minimizes the quadratic cost function $e(w)$, we take the gradient with respect to w and equate it to zero.

$$\nabla e(w) = -r + R w = 0$$

$$R w = r$$

$$w_{opt} = R^{-1} r$$

2. Calculate Inverse of Matrix R:

$$R = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 & 0.8182 \\ 0.8182 & 1 \end{bmatrix}$$

Determinant of R (Δ):

$$\Delta = (1)(1) - (0.8182)(0.8182)$$

$$\Delta = 1 - 0.66945 = 0.33055$$

Inverse Matrix R^{-1} :

$$R^{-1} = \frac{1}{\Delta} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \frac{1}{0.33055} \begin{bmatrix} 1 & -0.8182 \\ -0.8182 & 1 \end{bmatrix}$$

3. Calculate Optimum Weight Vector w :

$$w = R^{-1}r$$

$$w = \frac{1}{0.33055} \begin{bmatrix} 1 & -0.8182 \\ -0.8182 & 1 \end{bmatrix} \begin{bmatrix} 0.8182 \\ 0.354 \end{bmatrix}$$

Matrix Multiplication:

$$w = \frac{1}{0.33055} \begin{bmatrix} (1 \cdot 0.8182) + (-0.8182 \cdot 0.354) \\ (-0.8182 \cdot 0.8182) + (1 \cdot 0.354) \end{bmatrix}$$

Computing elements:

- Row 1: $0.8182 - 0.2896 = 0.5286$
- Row 2: $-0.6695 + 0.354 = -0.3155$

$$w = \frac{1}{0.33055} \begin{bmatrix} 0.5286 \\ -0.3155 \end{bmatrix}$$

Final Division:

$$w_1 = \frac{0.5286}{0.33055} \approx 1.599$$

$$w_2 = \frac{-0.3155}{0.33055} \approx -0.954$$

$w_{\text{opt}} \approx \begin{bmatrix} 1.60 \\ -0.95 \end{bmatrix}$

***** **5 b) Learning Rate Range for LMS Convergence****

Question: The Correlation Matrix R_x of the input vector $x(n)$ in the LMS algorithm is defined by $R_x = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$. Define the range of the values for the learning rate η (or α) of the LMS algorithm for it to be convergent in the mean square.

Solution:

To ensure the convergence of the LMS algorithm (specifically, convergence in the mean square), the learning rate η must satisfy the following condition related to the eigenvalues of the correlation matrix R_x :

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

Where $\lambda_{\max}$ is the maximum eigenvalue of the input correlation matrix R_x .

Step 1: Find the Eigenvalues of R_x

The characteristic equation is given by $|R_x - \lambda I| = 0$.

$$\det \begin{bmatrix} 1-\lambda & 0.5 \\ 0.5 & 1-\lambda \end{bmatrix} = 0$$

$$(1-\lambda)(1-\lambda) - (0.5)(0.5) = 0$$

$$(1-\lambda)^2 - 0.25 = 0$$

$$(1-\lambda)^2 = 0.25$$

Taking the square root of both sides :

$$1-\lambda = \pm 0.5$$

Solving for λ :

$$1 - \lambda_1 = 0.5 \Rightarrow \lambda_1 = 0.5$$

$$1 - \lambda_2 = -0.5 \Rightarrow \lambda_2 = 1.5$$

So, the eigenvalues are $\lambda_1 = 0.5$ and $\lambda_2 = 1.5$.

The maximum eigenvalue is $\lambda_{\max} = 1.5$.

Step 2: Determine the Range for η

Substituting $\lambda_{\max}$ into the convergence condition:

$$0 < \eta < \frac{2}{1.5}$$

$$0 < \eta < \frac{2}{3/2}$$

$$0 < \eta < \frac{4}{3}$$

$\mathbf{0} < \eta < 1.333$